

面向代码变更的 Playwright 端到端测试自动修复研究综述

——兼论变更约束下的端到端用例补充生成

学位论文文献综述

2026 年 6 月 17 日

摘要

在敏捷开发与持续集成持续交付的工程实践中，Web 应用以提交与合并请求为单位高频演化，使端到端（End-to-End, E2E）测试因定位符断裂与断言失配而频繁失效，维护负担居各类测试之首。大语言模型（Large Language Model, LLM）的兴起把测试修复从基于规则的自愈推向了语义驱动自动修复，并催生了以代码变更（Code Diff）为触发与约束的测试演化方向。本文以“修复”为主线、以“变更约束生成”为辅线，系统梳理变更感知测试演化、Web 与 E2E 测试修复与自愈、E2E 稳定性与失效分型、变更驱动的用例补充生成、受影响测试定位的工程支撑，以及评测资源与可信验证六个主题的代表性进展。综述发现：变更感知修复的范式已较成熟却几乎全部落在单元与回归测试层，Web 与 E2E 修复多由运行时失败触发而未把引发断裂的源码变更显式作为修复上下文，且普遍缺乏“有意改动与真回归”的判别，导致误报偏高；面向 Playwright 的可复现评测资源直到近期才出现。由此可见，以代码变更为一等上下文、面向 Playwright 的低误报且可解释的 E2E 自动修复仍是明确空白，本文据此提炼出可检验的研究议程与评测路线。

1 引言

软件测试是质量保障的核心活动，而测试代码本身也是需要随产品代码持续演化的“活制品”。在现代 Web 应用中，端到端测试通过在真实浏览器内模拟用户的完整业务流程，验证跨组件、跨页面的集成行为，是发现集成层缺陷、保障真实用户体验的关键手段；Playwright、Selenium、Cypress 等框架构成了当前 Web 自动化测试的主流工具链。然而，E2E 测试长期受困于三类相互交织的痛点：脚本编写需精确刻画元素定位与交互时序，维护成本高；前端结构、组件层级与接口的细微变更即可令定位符（locator）断裂或断言失配，对演化高度敏感；异步加载与网络时延带来的非确定性失败（flaky）进一步侵蚀其可信度 [1, 2]。已有实证表明，相当比例的失效可归因于“过时而未及时更新的测试套件”，这使“如何让测试随代码演化而自动修复”成为亟待解决的工程问题 [3]。

与此同时，大语言模型为软件测试注入了新动能。面向测试的权威综述系统勾勒了 LLM 在测试生成、缺陷复现与测试预言等环节的版图与愿景 [4]；面向 Web 测试的最新综述进一步指出，人工智能尤其是 LLM 正在重塑 Web 测试的研究与工业实践 [2]。在持续集成成为常态的背景下，“以代码变更为单位驱动测试演化”逐渐取代“整套重跑”的粗放模式，成为兼顾成本与有效性的关键路径。本文的立足点正是这一交叉地带：以引发失效的代码变更为线索，针对 Playwright

E2E 测试开展自动修复，并把变更约束下的用例补充生成作为闭环的补全环节。

需要说明的是，本文的论域与传统“自愈定位”（self-healing locator）有本质差异。自愈方法多在运行时依据新旧文档对象模型（Document Object Model, DOM）做元素重匹配，并不关心“为什么会断裂”；而变更驱动的修复试图把“引发断裂的源码差异”作为一等输入，从而获得因果线索、抑制幻觉并支持“是否应当如实报告回归”的判断。围绕这一主线，本文依次展开六个主题：变更感知的测试演化、Web 与 E2E 测试修复与自愈、E2E 稳定性与失效分型、变更驱动的用例补充生成、受影响测试定位的工程支撑，以及评测资源与可信验证；随后在讨论与展望中归纳异质性、方法学陷阱与可检验的研究议程。

2 变更感知的测试演化：从单元回归到端到端的范式迁移

本主题确立全篇的方法论坐标：变更感知测试演化已经形成清晰范式，但其重心仍停留在单元与回归测试层，这恰是把范式迁移至 E2E 的机会所在。

变更感知修复的代表性范式由若干工作共同奠定。UTFix 与本文主线最为贴近：当焦点方法发生变更后，它借助静态切片、动态切片与失败信息上下文驱动 LLM 修复对应单元测试，分别应对“断言失败”与“覆盖率下降”两类问题，并在演化中的 Python 项目上取得了可观的断言修复率 [3]。其“以切片获取最小相关上下文、以失败信息驱动修复”的设计思想，可直接迁移为“以 diff 切片加失败 trace 构造 E2E 修复上下文”。TaRGET 则把测试用例修复建模为“语言翻译”任务并微调代码模型，构建了规模可观的修复基准，证明了序列到序列框架在测试修复上的有效性 [5]。Fix the Tests 用静态上下文收集器配合神经重排序器，修复因代码变更未同步而过时的测试，强调“收集—排序—修复”的流水线在大型项目上的准确性收益 [6]。Unit Test Update 进一步把“修复失效测试”与“增强测试以验证新功能”统一在一个 LLM 驱动的上下文收集与错误类型感知精炼框架内，与 UTFix 形成互补 [7]。YATE 把“修复”嵌入“生成”闭环，指出生成阶段产出的含错测试经轻量修复即可重获价值，提示修复与生成本就是一体两面 [8]。

在变更驱动“生成”一侧，路径同样清晰。SymPrompt 面向回归场景，按被测方法的执行路径将生成拆解为多阶段提示并注入类型与依赖上下文，在无需训练的条件下显著提升覆盖率，其“执行路径感知加上下文注入”策略可迁移到“以变更路径引导 E2E 用例生成” [9]。一组工作把“变更行到定向测试”的范式逐步做实：有研究针对一次 commit 变更直接用 LLM 生成回归测试以暴露该变更引入的缺陷 [10]；ChaCo 不追求整体覆盖率，而是精准定向合并请求中仍未覆盖的“最后一公里”变更行生成补充测试，正对应“用 diff 决定生成哪些测试” [11]；亦有工作系统讨论了合并请求粒度测试生成的可行性与评测差距，为 PR 粒度的方法评估提供了方法论参照 [12]；工业侧的实践以“应当在合并前失败的捕获型测试”为目标，发现变更感知方法相较硬化型测试在候选捕获上具有数倍优势，并以规则加模型的方式抑制误报 [13]。TestWeaver 则集成轻量程序分析构造聚焦执行上下文、以反馈驱动迭代突破覆盖率停滞 [14]，Code-A1 探索代码模型与测试模型的对抗式协同进化以拓展白盒测试生成思路 [15]。

尤为关键的是“有意改动与真回归”的判别。Testora 首次把“代码变更意图（自然语言）”与“变更导致的行为差异”加以对比，从而区分“有意行为改变”与“真回归”，避免把所有差异都误报为失败 [16]。这一思想对 E2E 断言与期望的更新判定尤为重要：当 UI 行为按预期改变时，正确的

处理是更新期望而非“修回旧行为”。综上，变更感知演化的方法骨架已经稳固，但其落点几乎全在代码级单元与回归测试，输入多为后端代码 diff，前端与 E2E 层面仍属空白；演化场景下的实证亦提示，生成与修复的有效性会随演化而衰减，需以反馈与验证加以稳固 [17]。把这一骨架迁移到 Playwright E2E，正是本文主线的逻辑起点。

3 Web 与端到端测试修复与自愈：现状、技术栈与局限

本主题直接对标“修复”任务，目的是定位现有 Web 与 E2E 修复在技术栈与输入信号上的具体短板。

在与本文技术栈最贴近的工作中，自治多智能体的 UI 测试修复案例尤为重要：它以 LLM 编排配合 Playwright 执行与检索增强知识库，构建面向企业级测试套件的自治修复系统，应对每屏数百个动态 UI 元素，并展示了从人工指导逐步走向高自治的现实边界 [18]。该工作验证了“LLM 加编排加 Playwright 加检索”的工程可行性，是本文头号对标与基线，但它聚焦“自治修复”的可达性，并未将引发断裂的代码变更作为核心输入信号。面向 Web 演化致脚本断裂，语义层面的测试修复较早地把“理解页面语义而非字面匹配”引入修复回路 [19]。在元素重定位方向，针对 Web UI 测试的工作先用既有技术做元素初匹配，再由模型完成后续匹配与断点修复，并设计“解释一致性校验器”抑制幻觉，强调属性优先级在重定位中的作用 [20]；这一“多属性优先级匹配加解释一致性校验”的组合，对定位符断裂修复具有直接借鉴意义。视觉视角的工作则用目标检测与图结构跨版本识别 UI 元素对应关系与“有意义的变更”，为非脚本化的 UI 变更检测提供了补充手段 [21]。

把视野放宽到测试维护的整体生态，可见自愈与 LLM 集成的现实图景。面向 Web 自动化的自愈框架综述刻画了“自愈定位”的工程脉络及其在脆弱性面前的局限 [22]；对人工智能辅助测试自动化的多源文献梳理系统盘点了商业工具与学术方法的现状与缺口 [23]；针对工业测试维护流程中 LLM 集成的探索性研究，则从真实流程视角揭示了落地约束与协作模式 [24]。修复技术亦可借力自动程序修复（Automated Program Repair, APR）的积累：GAMMA 把模板修复转化为掩码预测，用预训练模型直接产出补丁 [25]；分层知识注入把缺陷层、仓库层与项目层上下文逐级供给模型以提升修复率 [26]；从可追溯性视角理解修复智能体行为的实证，为 Agent 化修复的可解释与可控提供了洞见 [27]；StubCoder 关注桩代码的生成与修复，提示“可执行依赖”的自动补全同样属于修复的边界问题 [28]。综合来看，修复方向技术已较成熟，“切片或上下文收集、失败驱动、幻觉抑制”是反复出现的共性要素，且已出现与 Playwright 一致的自治系统；但现有 E2E 与 UI 修复大多由运行时失败触发，鲜见以代码变更为先验信号定位“将要断裂”的用例并主动修复，更少把“引发断裂的 diff”显式作为修复上下文。这一短板正是本文的核心切入点。

4 端到端测试稳定性与失效分型：定位符断裂、异步 flaky 与脆弱性

修复之前必先甄别失效成因。本主题为“修复”提供精细的问题分型，并指出可复现评测资源的最新进展。

失效成因大体可分为三类。其一是定位符断裂：当被测应用结构变化导致定位符找不到目标元

素时，即便功能未变，测试也会断裂。ReproBreak 直接面向 Cypress 与 Playwright，通过分析数百个开源仓库识别含定位符变更的提交，并在变更最集中的项目上复现出数百个可复现的定位符断裂样本，附带自动复现脚本，且明确区分“致断裂的结构性变更”与“逻辑性变更”，使断裂与提交相互绑定 [29]。这为“代码变更致 E2E 失效到修复”的问题刻画与评估提供了难得的现成基准。其二是异步等待型 flaky: WEFix 利用浏览器 UI 变化预测客户端执行，自动生成正确的显式等待修复代码，在保持低开销的同时取得很高的修复正确率 [30]；亦有工作从时序角度自动修复异步等待型 flaky [31]。FlakyFix 用模型预测 flaky 的修复类别并据此修复测试代码 [32]；FlakyGuard 把代码视为图结构、用选择性图探索定位最相关上下文，缓解模型修复 flaky 时“上下文过少或过多”的两难，并完成工业规模评估 [33]。其三是脆弱性预测：有工作从预测视角出发，提前定位易断裂的 E2E 用例 [34]，这与“用 diff 预测受影响、将断裂用例”的思路天然契合。

由此可得两点对修复有直接意义的边界判断。一方面，flaky 失效与“真实变更致失效”的成因不同，修复前必须先做成因甄别，否则会把异步抖动误判为定位符断裂、施以错误修复；另一方面，可复现的、与提交绑定的定位符断裂数据集 [29] 使“变更上下文是否提升修复成功率”的对照实验成为可能，这是过去 Web 修复研究普遍缺乏的评测抓手。换言之，稳定性研究既提供了失效分型，又提供了可复现的实验地基，二者共同支撑“以 diff 为上下文的定向修复”从设想走向可检验。

5 变更驱动的端到端用例补充生成

作为修复主线的闭环补全，本主题考察 E2E 生成的可行路径，并指出其与代码变更解耦的共性缺陷。

LLM 生成 E2E 用例的可行性已被多方证实，关键在于如何把复杂页面与业务流程有效喂给模型。AutoE2E 自动推断 Web 应用的功能特性并转化为语义连贯、可执行的 E2E 用例，同时提出衡量功能覆盖率的基准，是该方向的代表作与重要评测参照 [35]。在页面与导航建模上，屏幕跳转图方法用“图加模型”刻画站点导航与复杂表单交互 [36]，VISCA 则把网页转化为层次化、语义丰富的组件抽象以优化模型的页面上下文输入 [37]；这两种“页面上下文表征”思路，对“如何把受变更影响的 UI 区域结构化地提供给模型”具有直接借鉴价值。在场景与意图驱动上，有工作以业务场景引导模型生成贴近真实用户流程的测试序列 [38]，亦有方法从自然语言描述直接生成可执行 E2E 脚本以填补“缺乏 E2E”的空白 [39]；ViMoTest 借助视图模型与投影式领域专用语言解耦表现层逻辑与 GUI 框架，意在降低 E2E 的规约成本、维护难度与 flaky [40]；面向“可塑移动应用”的工作则体现了“需求或变更到验证测试”的相似闭环 [41]。

然而，上述生成工作几乎都从“需求或当前页面状态”出发，未以代码变更作为生成的触发与约束。这意味着它们难以定向补充“本次变更引入但尚无覆盖”的新行为或新路径。把“代码变更到受影响 UI 流到 E2E 用例”这条链路打通，并借鉴覆盖率导向的多阶段提示 [9] 与“变更行定向”的思想 [11]，正是变更驱动生成区别于通用 E2E 生成的关键。需要强调的是，在本文的整体定位中生成是修复主线的补全环节：它与修复共享“受变更影响定位”与“执行反馈验证”两个机制，但实验规模相对更小，重在验证“以 diff 为约束相较无约束生成”在变更相关性上的提升。

6 受影响测试定位的工程支撑：回归测试选择与变更影响分析

无论修复还是补充生成，前提都是“基于 diff 精准定位受影响的用例”。本主题考察回归测试选择 (Regression Test Selection, RTS) 与变更影响分析 (Change Impact Analysis, CIA) 能为这一定位层提供哪些可迁移的能力。

在精确选择方面，基于语义修改推理的方法通过判别“是否真正改变语义”来更精确地选择受影响测试，从源头抑制过度选择 [42]；统一回归测试把变更代码与变更配置纳入同一选择框架 [43]；文件与方法依赖混合的选择在精度与效率间取得平衡 [44]；面向测试驱动开发场景的工作探索了该范式下的选择策略 [45]；针对动态类型语言调用图不准的难题，基于细粒度依赖分析的 Python 回归测试选择安全而有效地只执行受影响测试 [46]。在效率与流水线层面，增量代码覆盖率分析仅执行受变更影响的最小测试子集来更新覆盖率，显著提速并弥补了选择与覆盖率分析不兼容的问题 [47]；流水线感知的回归测试优化以强化学习区分合入前与合入后的不同目标 [48]；亦有工作把敏捷与持续集成下的“持续回归测试”形式化为按时间排序的构建链并定义“回归测试窗口” [49]。在变更影响分析方面，针对微服务系统的系统综述梳理了该领域全貌 [50]，而基于 Datalog 的语言无关分析适配了微服务的跨语言场景 [51]。

这些工作的共同价值在于：其细粒度依赖分析与语义变更推理思想，可被改造为“diff 到受影响 UI 流、E2E 用例”的定位机制，从而把修复与生成的作用范围收敛到真正受影响的用例上，降低无效计算与误报。但需指出其边界：现有 RTS 与 CIA 几乎都面向代码级单元与集成测试，尚未延伸到前端 UI 流与 E2E 脚本的影响定位；如何把“源码 diff”映射到“页面区域与用户流”，仍是需要新方法填补的环节。

7 评测资源、工业实践与可信验证

可靠评测与可信验证是方法研究的前提。本主题盘点可用的基准与数据集、工业落地经验，以及抑制幻觉的验证门槛，并据此说明本文方法的评测可行性。

在基准与数据集方面，面向定位符断裂的可复现数据集与 Playwright 技术栈直接契合，是修复评测的核心抓手 [29]；真实开源项目的 E2E 测试数据集汇集了数百个仓库、数万条 Web GUI 测试，可作为挖掘真实 Playwright 样本、支撑生成与泛化实验的语料库 [52]；面向真实缺陷修复的测试生成基准把测试生成与软件演化连接起来 [53]；演化场景下的测试质量评估提供了“覆盖率下降、漏检回归”等维度的评估框架 [17]。这些资源使“引入变更上下文是否提升修复成功率、是否降低误报”等问题可被量化检验。

在工业实践与可信验证方面，工业级 LLM 改进既有测试的实践以“可编译、可通过、能提升覆盖”为硬性门槛，体现了质量约束下的务实路线 [54]；从运行时“雕刻”单元测试并在持续集成中规模化执行的工作展示了规模化落地的可行性 [55]；对持续集成中 E2E 采用率的实证揭示了 E2E 在流水线中的现实门槛 [56]；对“智能体提交的合并请求是否包含测试代码”的研究，则反映了智能体时代测试供给的新趋势 [57]。在领域基础上，LLM 单元测试生成的奠基性实证刻画了模型在编译、覆盖率与可读性上的能力边界 [58]；搜索式方法在停滞时调用模型逃逸覆盖率瓶颈的工作把搜索与生成结合 [59]；以变异测试引导生成更具揭错能力的测试 [60]、把模型与搜索式测试

系统对比 [61]、提供生成校验修复闭环框架 [62]、在代码与测试对齐语料上训练模型 [63]，以及把测试脚本生成与迁移引入移动端 [64]，共同构成了方法基石。综述层面的工作则为背景与相关工作提供骨架：包括视觉 GUI 测试综述 [65] 与人工智能驱动的程序修复与代码生成综述 [66]。可信验证的共性经验是：以真实执行加解释一致性校验作为修复结果的双重门槛，并以反馈迭代收敛，这一原则可直接迁移到 Playwright 修复回路。

8 讨论

综合六个主题，可以提炼出若干跨研究的异质性与方法学陷阱。

第一，研究落点存在系统性错位。变更感知工作“懂 diff 但不做 E2E”，其输入多为后端代码 diff、评估以覆盖率为主 [9–11]；E2E 工作“做 E2E 但不懂 diff”，其触发多来自需求或运行时失败、与代码变更解耦 [18, 35, 36]。二者的交集——以代码变更为触发与约束，自动判定受影响的 E2E 用例并对其修复与补充生成——在现有工作中尚未被专门研究，这正是本文价值所在。

第二，输入信号与因果性的缺位。多数 Web 与 E2E 修复仅依赖新旧 DOM 做元素重匹配 [20, 22]，缺乏“引发断裂的源码差异”这一因果线索，容易在结构性重构面前产生幻觉式修复。把 diff 切片纳入上下文，并以解释一致性与真实执行双重验证 [3, 20]，是抑制幻觉的可行方向。

第三，误报来源的判别普遍薄弱。当 UI 行为按预期改变时，“修回旧行为”本身就是错误；只有引入“有意改动与真回归”的判别 [16]，才能在“更新期望、修复脚本、如实报告回归”三者间正确分流。这一判别是变更驱动修复区别于通用自愈的关键差异化。

第四，评测口径的异质性与可复现性风险。不同工作在数据集、指标与基线上差异较大，部分前沿成果仍处预印本阶段、缺乏开源实现，复现存在不确定性 [18]。所幸面向 Playwright 的可复现定位符断裂数据集 [29] 与真实 E2E 语料 [52] 的出现，使“以 diff 为上下文”的对照实验具备了可复现地基；同时应清醒认识到，把“测试侧定位符变更提交”关联到“应用侧引发断裂的 diff”仍需要额外的工程抽取，个别样本可能退化为“新旧 DOM 加 trace”的上下文。

第五，稳定性与修复的耦合。flaky 与真实变更致失效成因不同 [30, 33]，若不先做成因甄别，修复极易南辕北辙；因此在 E2E 修复回路中嵌入失效分型，是保证修复有效性的前置条件。

9 展望

基于上述判断，可提出一条以修复为重心、以生成为补全的可检验研究议程。

其一，把“引发失效的代码变更”确立为 E2E 修复与生成的一等上下文与触发器。具体而言，解析提交或合并请求的 diff，结合失败 trace、报错堆栈与失效时刻的 DOM 快照及新旧定位符，构造“引发该 E2E 失效的最小变更上下文”，其切片与上下文收集思想可借鉴单元测试修复的成熟做法 [3, 6]。该上下文构造既是方法核心，也是最大工程难点，可作为首要研究内容。

其二，建立“判别先行、分支处理”的修复流水线。先以失效分型区分 flaky 与变更致失效 [33]，再以变更意图判别区分有意改动与真回归 [16]，据此在“更新期望、修复脚本、报告回归”之间分流；

对定位符断裂采用多属性优先级匹配加语义定位重写 [20]，对断言失配结合变更语义更新期望，并统一以解释一致性校验加 Playwright 真实执行验证、以执行反馈迭代收敛。

其三，把受影响定位从代码级延伸到 UI 流级。将 RTS 与 CIA 的细粒度依赖与语义变更推理 [42, 46, 51] 改造为“diff 到受影响 UI 流、E2E 用例”的映射，并借助页面语义抽象 [36, 37] 把受影响的 UI 区域结构化，从而把修复与补充生成收敛到真正受影响的用例。

其四，把变更驱动从修复延伸到按需生成。针对变更引入但尚无覆盖的新行为，以覆盖率导向的多阶段提示与变更行定向 [9, 11] 生成语义连贯、可执行的 Playwright 用例，并以执行反馈迭代收敛；重在验证“以 diff 为约束”相较无约束生成在变更相关性上的提升 [35]。

其五，建立可复现、面向 Playwright 的评测协议。以可复现定位符断裂数据集作为修复主基准 [29]，以真实开源 E2E 语料支撑泛化与持续集成开销评估 [52]，参照演化质量评估框架 [17] 与工业级误报抑制门槛 [54]，从修复成功率、精确匹配率、误报率、迭代次数与可解释性等维度综合衡量。需要正视的现实约束包括：模型的幻觉与不稳定、前沿基线的可复现性，以及 diff 到失效关联可能缺失等风险，对应的应对是双重验证、设置纯属性匹配与无 diff 纯模型修复的兜底基线，以及在关联缺失时退化为新旧 DOM 加 trace 的上下文。

10 结论

本文以“修复”为主线、以“变更约束生成”为辅线，系统综述了变更感知测试演化、Web 与 E2E 测试修复与自愈、E2E 稳定性与失效分型、变更驱动的用例补充生成、受影响测试定位的工程支撑，以及评测资源与可信验证六个主题的代表性进展。综述表明：变更感知修复范式已较成熟却几乎全部落在单元与回归测试层；Web 与 E2E 修复多由运行时失败触发，未把引发断裂的源码变更显式作为修复上下文，且普遍缺乏“有意改动与真回归”的判别而导致误报偏高；E2E 生成虽已可行却与代码变更解耦；面向 Playwright 的可复现评测资源直到近期才出现。由此，“以代码变更为一等上下文、面向 Playwright 的低误报且可解释的 E2E 自动修复，并以变更约束生成补全闭环”仍是明确而有价值的研究空白。本文据此提出的“变更上下文构造、判别先行的修复流水线、UI 流级受影响定位、变更约束补充生成、可复现评测协议”研究议程，可作为后续方法设计与实验评估的起点。

参考文献

- [1] Leotta M, others. Challenges of End-to-End Testing with Selenium WebDriver and How to Face Them: A Survey[C/OL] // International Conference on Software Testing, Verification and Validation (ICST). 2023. <https://doi.org/10.1109/ICST57152.2023.00039>.
- [2] Anon. A Survey on Web Testing: On the Rise of AI and Applications in Industry[Z/OL]. 2025. <https://arxiv.org/abs/2503.05378>.
- [3] Rahman S, Kuhar S, Cirisci B, Garg P, Wang S, Ma X, Deoras A, Ray B. UTFix: Change Aware Unit Test Repairing using LLM[J/OL]. Proceedings of the ACM on Programming Languages (OOPSLA), 2025. <https://arxiv.org/abs/2503.14924>.

- [4] Wang J, Huang Y, Chen C, Liu Z, Wang S, Wang Q. Software Testing with Large Language Models: Survey, Landscape, and Vision[J/OL]. IEEE Transactions on Software Engineering, 2023. <https://arxiv.org/abs/2307.07221>.
- [5] Yaraghi A S, Holden D, Kahani N, Briand L. Automated Test Case Repair Using Language Models[J/OL]. IEEE Transactions on Software Engineering, 2025. <https://arxiv.org/abs/2401.06765>.
- [6] Anon. Fix the Tests: Augmenting LLMs to Repair Test Cases with Static Collector and Neural Reranker[C/OL] // International Symposium on Software Reliability Engineering (ISSRE). 2024. <https://arxiv.org/abs/2407.03625>.
- [7] Anon. Unit Test Update through LLM-Driven Context Collection and Error-Type-Aware Refinement[C/OL] // Automated Software Engineering (ASE). 2025. <https://arxiv.org/abs/2509.24419>.
- [8] Anon. YATE: The Role of Test Repair in LLM-Based Unit Test Generation[Z/OL]. 2025. <https://arxiv.org/abs/2507.18316>.
- [9] Ryan G, Jain S, Shang M, Wang S, Ma X, Ramanathan M K, Ray B. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM[J/OL]. Proceedings of the ACM on Software Engineering (FSE), 2024. <https://arxiv.org/abs/2402.00097>.
- [10] Liu J, others. Can LLMs Generate Regression Tests for Software Commits?[Z/OL]. 2025. <https://arxiv.org/abs/2501.11086>.
- [11] Anon. Change And Cover: Last-Mile, Pull Request-Based Regression Test Augmentation[Z/OL]. 2026. <https://arxiv.org/abs/2601.10942>.
- [12] Anon. PR-Aware Automated Unit Test Generation: Challenges and Opportunities[Z/OL]. 2026. <https://arxiv.org/abs/2605.25285>.
- [13] Anon. Just-in-Time Catching Test Generation at Meta[Z/OL]. 2026. <https://arxiv.org/abs/2601.22832>.
- [14] Anon. TestWeaver: Execution-Aware, Feedback-Driven Regression Test Generation with Large Language Models[Z/OL]. 2025. <https://arxiv.org/abs/2508.01255>.
- [15] Anon. Code-A1: Adversarial Co-Evolution of Code LLM and Test LLM via Reinforcement Learning[Z/OL]. 2026. <https://arxiv.org/abs/2603.15611>.
- [16] Pradel M. Testora: Using Natural Language Intent to Detect Behavioral Regressions[Z/OL]. 2025. <https://arxiv.org/abs/2503.18597>.
- [17] Anon. Evaluating LLM-Based Test Generation Under Software Evolution[Z/OL]. 2026. <https://arxiv.org/abs/2603.23443>.
- [18] Anon. Practical Limits of Autonomous Test Repair: A Multi-Agent Case Study[Z/OL]. 2026. <https://arxiv.org/abs/2605.01471>.
- [19] Qi X, others. Semantic Test Repair for Web Applications[C/OL] // ESEC/FSE. 2023. <https://doi.org/10.1145/3611643.3616324>.
- [20] Xu Z, Li Q, Tan S H. Understanding and Enhancing Attribute Prioritization in Fixing Web UI Tests with LLMs[C/OL] // International Conference on Software Testing, Verification and Validation (ICST). 2025. <https://arxiv.org/abs/2312.05778>.

- [21] Moradi M, others. AI for Context-Aware Visual Change Detection in Software Test Automation[J/OL]. Progress in Artificial Intelligence, 2024. <https://arxiv.org/abs/2405.00874>.
- [22] Anon. Effortless Test Maintenance: A Critical Review of Self-Healing Frameworks[J/OL]. International Journal for Research in Applied Science and Engineering Technology (IJRASET), 2023. <https://doi.org/10.22214/ijraset.2023.56048>.
- [23] Anon. A Multivocal Literature Review on the Use of AI for Software Testing[J/OL]. Information and Software Technology, 2024. <https://arxiv.org/abs/2408.06224>.
- [24] Anon. Exploring the Integration of LLMs in Industrial Test Maintenance Processes[Z/OL]. 2024. <https://arxiv.org/abs/2409.06416>.
- [25] Zhang Q, Fang C, Zhang T, Yu B, Sun W, Chen Z. GAMMA: Revisiting Template-Based Automated Program Repair via Mask Prediction[C/OL] // Automated Software Engineering (ASE). 2023. <https://arxiv.org/abs/2309.09308>.
- [26] Anon. Hierarchical Knowledge Injection for Improving LLM-based Program Repair[C/OL] // Automated Software Engineering (ASE). 2025. <https://arxiv.org/abs/2506.24015>.
- [27] Anon. Understanding Automated Program Repair Agents Through the Lens of Traceability[Z/OL]. 2025. <https://arxiv.org/abs/2506.08311>.
- [28] Zhu H, Wei L, Terragni V, Liu Y, Cheung S-C, Wu J, Sheng Q, Mao B, Zhang L. StubCoder: Automated Generation and Repair of Stub Code for Mock Objects[J/OL]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2023. <https://arxiv.org/abs/2307.14733>.
- [29] Anon. ReproBreak: A Dataset of Reproducible Web Locator Breaks[Z/OL]. 2026. <https://arxiv.org/abs/2605.12158>.
- [30] Liu X, Song Z, Fang W, Zhang W, Wang W. WEFix: Intelligent Automatic Generation of Explicit Waits for Efficient Web End-to-End Flaky Tests[C/OL] // The Web Conference (WWW). 2024. <https://arxiv.org/abs/2402.09745>.
- [31] Anon. Automated Repair of Asynchronous Wait Flaky Tests in Web Testing[Z/OL]. 2023. <https://arxiv.org/abs/2305.08592>.
- [32] Fatima S, Hemmati H, Briand L. FlakyFix: Using Large Language Models for Predicting Flaky Test Fix Categories and Test Code Repair[J/OL]. IEEE Transactions on Software Engineering, 2024. <https://arxiv.org/abs/2307.00012>.
- [33] Anon. FlakyGuard: Automatically Fixing Flaky Tests at Industry Scale[C/OL] // Automated Software Engineering (ASE). 2025. <https://arxiv.org/abs/2511.14002>.
- [34] Starace L L L, Martino S D. Towards Predicting Fragility in End-to-End Web Tests[C/OL] // Evaluation and Assessment in Software Engineering (EASE). 2024. <https://doi.org/10.1145/3661167.3661179>.
- [35] Alian P, Nashid N, Shahbandeh M, Mesbah A. Feature-Driven End-to-End Test Generation[C/OL] // International Conference on Software Engineering (ICSE). 2025. <https://arxiv.org/abs/2408.01894>.
- [36] Anon. Automated Web Application Testing: End-to-End Test Case Generation with LLMs and Screen Transition Graphs[Z/OL]. 2025. <https://arxiv.org/abs/2506.02529>.
- [37] Anon. VISCA: Inferring Component Abstractions for Automated End-to-End Web Testing[Z/OL]. 2025. <https://arxiv.org/abs/2506.04161>.

- [38] Anon. Scenario-Guided LLM-based Mobile GUI Testing[J/OL]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2025. <https://arxiv.org/abs/2506.05079>.
- [39] Anon. GenIA-E2ETest: A Generative AI-Based Approach for End-to-End Test Automation[C/OL] // Brazilian Symposium on Software Engineering (SBES). 2025. <https://arxiv.org/abs/2510.01024>.
- [40] Anon. ViMoTest: Specifying ViewModel-Based GUI Test Scenarios using Projectional Editing[C/OL] // International Conference on Software and Computer Technologies (ICSCT). 2025. <https://arxiv.org/abs/2504.16753>.
- [41] Anon. Automated Functional Testing for Malleable Mobile Applications Driven from User Intent[Z/OL]. 2026. <https://arxiv.org/abs/2604.02079>.
- [42] Liu Y, Zhang J, Nie P, Gligoric M, Legunsen O. More Precise Regression Test Selection via Reasoning about Semantics-Modifying Changes[C/OL] // International Symposium on Software Testing and Analysis (ISSTA). 2023. <https://doi.org/10.1145/3597926.3598086>.
- [43] Wang S, others. Test Selection for Unified Regression Testing[C/OL] // International Conference on Software Engineering (ICSE). 2023. <https://doi.org/10.1109/ICSE48619.2023.00145>.
- [44] Chen Z, others. Hybrid Regression Test Selection by Integrating File and Method Dependences[C/OL] // Automated Software Engineering (ASE). 2024. <https://doi.org/10.1145/3691620.3695525>.
- [45] Anon. Regression Test Selection in Test-Driven Development[J/OL]. Automated Software Engineering Journal, 2023. <https://doi.org/10.1007/s10515-023-00405-w>.
- [46] Anon. Names Are All You Need: Regression Test Selection for Python[Z/OL]. 2026. <https://arxiv.org/abs/2605.25356>.
- [47] Wang J A, Wang K, Nie P. Efficient Incremental Code Coverage Analysis for Regression Test Suites[C/OL] // Automated Software Engineering (ASE). 2024. <https://arxiv.org/abs/2410.21798>.
- [48] Anon. Practical Pipeline-Aware Regression Test Optimization for Continuous Integration[C/OL] // International Conference on Software Testing, Verification and Validation (ICST). 2025. <https://arxiv.org/abs/2501.11550>.
- [49] Anon. Formalizing Regression Testing for Agile and Continuous Integration Environments[Z/OL]. 2025. <https://arxiv.org/abs/2511.02810>.
- [50] Anon. Change Impact Analysis in Microservice Systems: A Systematic Literature Review[J/OL]. Journal of Systems and Software, 2024. <https://doi.org/10.1016/j.jss.2024.112241>.
- [51] Shi Q, others. Datalog-Based Language-Agnostic Change Impact Analysis for Microservices[C/OL] // International Conference on Software Engineering (ICSE). 2025. <https://doi.org/10.1109/ICSE55347.2025.00115>.
- [52] Pontillo V, others. E2EGit: A Dataset of End-to-End Web Tests in Open Source Projects[C/OL] // International Conference on Mining Software Repositories (MSR). 2025. <https://doi.org/10.1109/MSR66628.2025.00121>.
- [53] Mundler N, Muller M N, He J, Vechev M. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents[C/OL] // Advances in Neural Information Processing Systems (NeurIPS). 2024. <https://arxiv.org/abs/2406.12952>.

- [54] Alshahwan N, Chheda J, Finogenova A, Gokkaya B, Harman M, Harper I, Marginean A, Sengupta S, Wang E. Automated Unit Test Improvement using Large Language Models at Meta[C/OL] // FSE Companion. 2024. <https://arxiv.org/abs/2402.09171>.
- [55] Alshahwan N, Harman M, Marginean A, Tal R, Wang E. Observation-Based Unit Test Generation at Meta[C/OL] // FSE Companion. 2024. <https://arxiv.org/abs/2402.06111>.
- [56] Anon. Android Instrumentation Testing in CI: Practices, Patterns, and Performance[Z/OL]. 2026. <https://arxiv.org/abs/2604.03438>.
- [57] Anon. Do Autonomous Agents Contribute Test Code? A Study of Tests in Agentic Pull Requests[Z/OL]. 2026. <https://arxiv.org/abs/2601.03556>.
- [58] Schafer M, Nadi S, Eghbali A, Tip F. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation[J/OL]. IEEE Transactions on Software Engineering, 2023. <https://arxiv.org/abs/2302.06527>.
- [59] Lemieux C, Inala J P, Lahiri S K, Sen S. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models[C/OL] // International Conference on Software Engineering (ICSE). 2023. <https://doi.org/10.1109/ICSE48619.2023.00085>.
- [60] Dakhel A M, others. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing[J/OL]. Information and Software Technology, 2023. <https://arxiv.org/abs/2308.16557>.
- [61] Vikram V, others. An Empirical Study of Using ChatGPT for Unit Test Generation (ChatGPT vs SBST)[J/OL]. IEEE Transactions on Software Engineering, 2023. <https://arxiv.org/abs/2307.00588>.
- [62] Chen Y, others. ChatUniTest: A Framework for LLM-Based Test Generation[C/OL] // FSE Companion. 2023. <https://arxiv.org/abs/2305.04764>.
- [63] Rao N, Jain K, Alon U, Goues C L, Hellendoorn V J. CAT-LM: Training Language Models on Aligned Code and Tests[C/OL] // Automated Software Engineering (ASE). 2023. <https://arxiv.org/abs/2310.01602>.
- [64] Anon. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing[C/OL] // International Conference on Software Quality, Reliability and Security (QRS). 2023. <https://arxiv.org/abs/2309.13574>.
- [65] Yu S, Fang C, others. Vision-Based Mobile App GUI Testing: A Survey[J/OL]. ACM Computing Surveys, 2023. <https://arxiv.org/abs/2310.13518>.
- [66] Anon. A Comprehensive Survey of AI-Driven Advancements and Techniques in Automated Program Repair and Code Generation[Z/OL]. 2024. <https://arxiv.org/abs/2411.07586>.